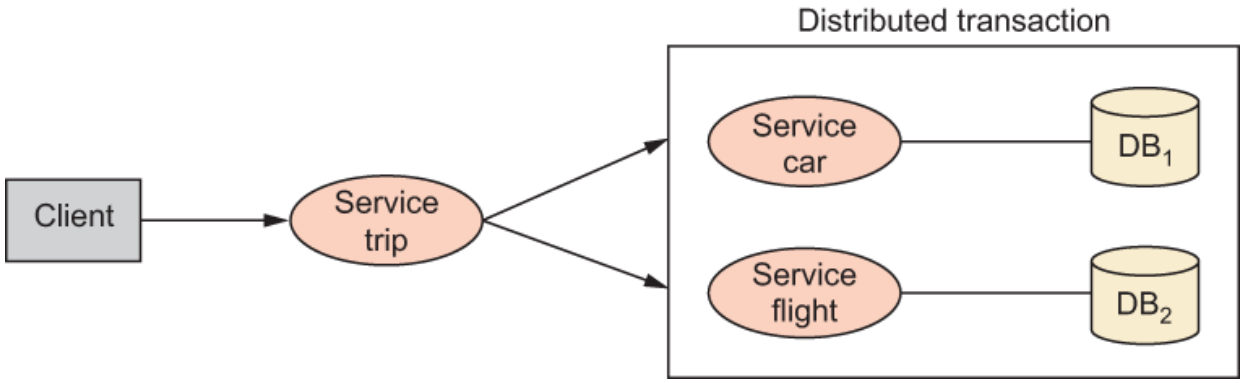

Figure B.5 Distributed transaction

In these scenarios, you can use a database pattern called Saga, although it would be better to avoid the transaction in the first place. Unlike a two-step commit, a Saga is not a blocking protocol. The key concept is to create a counteraction or compensation for each action required for the transaction. Thus, if an action fails, the transaction can be aborted by performing a counteraction for each successful action, and the final state is the same as the initial state before any change was committed.

CONSISTENCY

Now let's talk about consistency. It won't surprise you that replication of data leads to discrepancies between its copies. Whether the data is replicated in a database or a group of services, the fact that there are multiple instances of the same data presents the problem of consistency—or linearizability, to be precise (https://jepsen.io/consistency). Attempting to update the state of each instance of a piece of data takes time for all replicas to receive the message if they receive it.